Vendor

What it is

The vendor registry: the parties that send quotations and invoices. A vendor owns everything else in the product — its committed services are the rate card, its invoices reconcile against that rate card, and its **rules** steer the AI phases that read both. Everything in the app is scoped to a vendor.

The feature is deliberately small: a flat list, a four-field form, and a rules editor. There is no vendor profile page — see Routes.

Routes

Path	Purpose
`/vendor`	The registry. Every vendor, one table, no paging.
`/vendor/add`	Create a vendor.
`/vendor/$vendorId`	**That vendor's quotations** — not a vendor profile.
`/vendor/$vendorId/edit`	Edit a vendor.
`/vendor/$vendorId/rules`	That vendor's rules.

The route's loader fetches the vendor solely to title the page (`${vendor.name} Quotations`). A vendor's `code` , `location` and `comments` are visible in exactly two places — the list table and the edit form. Nothing else ever renders them.

Only `/vendor/$vendorId` carries search params (`page` , `pageSize` , `q`), and they belong to the quotations table, not to the vendor.

Data

`GET /vendors` returns the lot. There is no pagination, filter, sort or search — on the server or the client.

Op	Method	Endpoint
List	GET	`/vendors`
One	GET	`/vendors/:id`
Types	GET	`/vendors/types`
Create	POST	`/vendors`
Update	PATCH	`/vendors/:id`
Delete	DELETE	`/vendors/:id`

Every mutation invalidates the `["vendors"]` key. Rules live under `["vendors", vendorId, "rules"]` and quotations under `["vendors", vendorId, "quotations"]` , so that one invalidation also refreshes the

cached rules and quotations under it.

Shape

```
export type Vendor = {
  id: string
  name: string
  code?: string | null
  type?: string | null
  location?: string | null
  comments?: string | null
  createdAt: string
  updatedAt: string
}
```

The form (`createVendorSchema`) requires `name` , `code` , `type` and `location` ; only `comments` is optional. `Vendor` is a hand-written type, not a zod schema, so responses are typed but not validated at runtime — only form input is.

`createVendorSchema` is reused verbatim for PATCH, so an edit always sends the full object.

Vendor type is free text, not an enum. The combobox suggests existing values from `GET /vendors/types` and offers `Create "{search}"` for anything new. This matters because the sidebar groups vendors by `type` — a typo makes a new group.

Screens

The list (`vendor-table.tsx`) — Name, Code, Type, Location, Comment, and a row menu. Clicking a row goes to that vendor's quotations; the actions cell stops propagation so the menu doesn't navigate. The menu has three items: Rules, Edit, Delete. Delete opens a confirm dialog before permanently deleting the vendor.

Add / edit (`add-vendor-form.tsx` , `edit-vendor-form.tsx`) — two explicit variants over a shared `VendorFields` body, each owning its own mutation and toasts. Both return to `/vendor` on save and on cancel.

Rules

A rule is a **natural-language guardrail** the customer writes to steer one AI phase for one vendor. The authoritative definition lives in `packages/shared/src/rules/rule.ts` :

> Vendor guardrails — natural-language instructions a customer sets per vendor to steer an LLM phase. Stored as :Rule nodes in Neo4j; the selected rules for a phase are appended to that phase's prompt. Distinct from a line item's reviewer `guidance` .
>
> The category is the phase the rule steers. It also implies the document type (service_* → source docs, line_item/matching → transaction docs), which is how a rule targets the right pipeline without a separate documentType field.

So the **category is the phase**, and it doubles as the document-type selector — which is why a rule has no `documentType` field.

Value	Label	Steers
`service_extraction`	Service Extraction	how services/rates are read out of quotations
`line_item_extraction`	Line Item Extraction	how billed lines are read out of invoices
`service_grouping`	Service Grouping	how similar services group as one
`matching_and_reconciliation`	Matching & Reconciliation	how billed lines are matched and reconciled

`RULE_CATEGORIES` is a local mirror of the shared constant, pinned by a test. Its array order is the select order, and `RULE_CATEGORIES[0]` is the add form's default.

Don't confuse a **rule** with a line item's **guidance**: a rule is vendor-wide and steers a phase; guidance corrects one line.

Endpoints

Op	Method	Endpoint
List	GET	`/vendors/:vendorId/rules` (optional `?search=`)
Create	POST	`/vendors/:vendorId/rules`
Update	PATCH	`/vendors/:vendorId/rules/:id`
Delete	DELETE	`/vendors/:vendorId/rules/:id`

The search term is part of the query key (`["vendors", id, "rules", search]`), so each term caches separately and one prefix invalidation clears every variant.

The page

A searchable accordion. One rule opens at a time, and the "Add Rule" editor is modelled as the same slot via a sentinel — `openRuleId` is either a rule's id or `NEW_RULE_ID`, so the add form and an open row are mutually exclusive by construction rather than by luck. The add form renders *after* the last rule, showing the position it will occupy.

A collapsed row shows an index badge, title, content and created date — **not the category**. You must expand a rule to see which phase it steers. Deleting also requires expanding: Delete lives inside the edit form, not on the row.

Search is server-side, debounced 300ms. The backend returns a Lucene relevance **top-K**, so the count beside the heading is the number of rules returned.

The `01, 02…` badges are the row's position in a newest-first list; they are not stored identifiers and shift as rules are added.

Getting there

Two entry points: the vendor row menu, and the quotations toolbar's "Add Rule" button, which opens the rules list. Rules aren't reachable from the sidebar.

File index

`features/vendors/`

File	Purpose
`components/vendor-table.tsx`	The registry table; owns the delete mutation and mounts its dialog
`components/vendor-fields.tsx`	Shared field body for the add/edit variants
`components/add-vendor-form.tsx`	Create variant — own mutation, defaults, actions
`components/edit-vendor-form.tsx`	Edit variant — same fields, own mutation
`components/vendor-type-combobox.tsx`	Free-text type picker over `/vendors/types`
`components/delete-vendor-dialog.tsx`	Pure confirm; the mount site owns the mutation
`services/vendors.ts`	All six queries/mutations
`store/delete-vendor-dialog.ts`	Delete-confirm open state + target
`schemas.ts`	`createVendorSchema` (reused for update) + the `Vendor` wire type

`features/rules/`

File	Purpose
`components/rules-page.tsx`	Page shell: header, search, list; owns the delete mutation
`components/rules-context.tsx`	`openRuleId` + create/update; exports `NEW_RULE_ID`
`components/rule-list.tsx`	Rows, empty state, and the trailing add form
`components/rule-row.tsx`	One collapsible row
`components/rule-fields.tsx`	Shared field body for the add/edit variants
`components/add-rule-form.tsx`	Create variant
`components/edit-rule-form.tsx`	Edit variant; hosts the Delete trigger
`components/delete-rule-dialog.tsx`	Pure confirm
`services/rules.ts`	List/create/update/delete + the search-aware query key
`store/delete-rule-dialog.ts`	Delete-confirm open state + target
`lib/rule-category.ts`	Local mirror of the four categories + display labels
`schemas.ts`	`ruleFormSchema`
`types.ts`	The `Rule` wire type